Algorithms for Applications	HW2: *Dynamic Programming, Randomization*
CS 5112 Fall 2025	**Due: October 1, 11:59 PM ET**

Homework Instructions: The homework contains mainly three types of tasks:

- **Coding.** You can find a template for all the coding problems inside the `problems.zip` in the module. Please use these templates, to ensure the files and functions have the same names that the autograder will check for.

  Inside each subproblem directory (for example, `problems/problem_1`) you will find a `test.py` file alongside the implementation files (`a.py`, `b.py`, etc.). These files contain some sample test cases for the different problems. These test cases are intended to help you test and debug your solution. However, note that Gradescope will run a more comprehensive test suite. Feel free to add additional test cases to further test your code.

  In order to run the tests for a particular problem n, navigate into `problems/problem_n` and run: `python3 test.py`

  You can run all available tests for the assignment by navigating to `problems` and running: `python3 -m unittest`

  If a problem requires you to submit code, it will be marked with a 💻.

- **Algorithm Description.** When a questions requires designing an algorithm, you should describe what your algorithm does in the writeup, in clear concise prose. You can cite algorithms covered in class to help your description. You should also argue for your algorithm's asymptotic run time or, in some cases when indicated in the problem, for the run time bounds of your implementation.

  If a problem requires a writeup, it will be marked with a 📝.

- **Empirical Performance Analysis.** Some questions may ask you to do an empirical performance analysis of one or more algorithms' runtime under different values of n. For these questions, you should generate at least ten test cases for the implementation, for various values of n (include both small and large instances). Then, measure the performance locally on your own system, by taking the median runtime of the implementation over ten or more iterations. Graph the resulting median run times. The x-axis should be instance size and the y-axis should be median run time. You can use any plotting library of your choice. If you have never plotted on Python before, this matplotlib turorial has some examples.

  If several subquestions in a problem ask you to provide performance analysis, please do them all in the same environment (same system and configuration), in order to have a more accurrate comparison between algorithms. You can include graphs for a same problem in the same plot, as long as the different graphs are properly labeled.

  For these types of questions, you will not need to submit the tests you generate or your code for benchmarking or plotting the algorithms. However, you should include the graphs you generate into the write-up. You should also write about what you observe from the analysis and potentially compare it to the complexity analysis. Finally, also include a description

of the environment (CPU, operating system and version, amount of memory) you did the testing on.

If a problem requires a plot, it will be marked with a  .

Submission Instructions: Hand in your solutions electronically on Gradescope. There are two active assignments for this problem set on Gradescope: one with the autograder, where you submit your code, and another one where you submit the write-up.

Coding Submission: You only need to submit the code for the specific functions we ask you to implement. While you will write additional code to generate the performance analysis graphs when required, you do not need to submit that code.

To submit to the autograder, please zip the `problems` directory we provided you with. Therefore, your zipped file should have the following structure:

```
problems
├── problem_1
│   ├── a.py
│   ├── b.py
│   └── ...
├── problem_2
│   └── a.py
└── ...
```

In order to ensure your code runs correctly, do not change the names of the files or functions we have provide you. Additionally, do not import external python libraries in these files, you don't need any libraries to solve the coding assignments. Finally, if you write any helper functions for your solutions, please include them in the same files as the functions that call them.

We have some public tests in the autograder to verify that all the required files are included, so you will be able to verify on Gradescope that your work is properly formatted shortly after submitting. We strongly encourage checking well ahead of the due date that your solutions work on the autograder, and seeking out assistance from the TAs should it not. We cannot guarantee being responsive the night the assignment is due.

Write-up Submission: Your write-up should be a nicely formatted PDF prepared using the LaTeX template found in the handout zip archive. The template already has every problem listed in it and a stub for each problem indicating whether a write-up is required for that problem. If you do not have previous experience using LaTeX, we recommend using Overleaf. It is an online LaTeX editor, where you can upload and edit the template we provide. Note that you will also have to upload the provided hw.sty and template.cls files.

Academic Integrity: You may use online sources or tools (such as code generation tools), but any tools you use should be explicitly acknowledged and you must explain how you used them. You are responsible for the correctness of submitted solutions, and we expect you to understand them and be able to explain them when asked by teaching staff.

Collaboration Policy: Collaboration (in groups of up to three students) is encouraged while solving the problems, but:

1. list the netids of those in your group;
2. you may discuss ideas and approaches, but you should not write a detailed argument or code outline together;
3. notes of your discussions should be limited to drawings and a few keywords; you must **write up the solutions and write code on your own**.

Problem 1.

In this problem, you will compute the maximum sum of non-adjacent elements, maxsum(x), in two settings: a list and a tree. The definition of adjacent will be specific to each setting. Additionally, for each setting you must first write out the subproblem and then implement the full dynamic programming algorithm.

(a) (*10 points*) (📝💻)

Given a sequence of integers $x = x_1, x_2, \ldots, x_n$, your goal is to compute the largest sum of non-adjacent elements of x, `maxsum_list`(x). Adjacent elements of the sequence are next to each other: x_i and x_{i+1}.

Identify the dynamic programming subproblem and write its solution here. Then implement `maxsum_list` in `problem_1/a.py`.

Code input and output format. The input is a Python list, and the output should be an integer. For example, maxsum_list$([2, 1, 3, 4]) = 6$.

(b) (*10 points*) (📝💻)

Given a tree with nodes that contain integers, your goal is to compute the largest sum of non-adjacent elements. Trees are represented as a set of vertices $v_i \in V$ and edges $(v_i, v_j) \in E \subseteq V \times V$. There are no loops and every vertex has at most a single parent. Adjacent elements of the tree are those that have an edge between nodes, i.e. $(v_i, v_j) \in E$. For convenience, define the children of a node to be $c(v_i) = \{v_j \mid (v_i, v_j) \in E\}$.

Identify the dynamic programming subproblem and write its solution here. Then implement `maxsum_tree` in `problem_1/b.py`.

Code input and output format. The input is a tree represented as a pair of dictionaries: the first dictionary maps node names to node values, and the second dictionary maps each vertex to a list of child vertices. The output should be an integer. For example, maxsum_tree$(\text{values} = \{2:2, 1:1, 3:3, 4:4\}, \text{adjacency} = \{2:[1], 1:[3], 3:[4]\}) = 6$.

Problem 2.

Given n points in a 2D plane, your goal is to find the two points that are closest together according to Euclidean distance.

(a) (*5 points*) (💻)

Implement a $\mathcal{O}(n^2)$ brute-force method. Include your implementation in `problem_2/a.py`.

(b) (*5 points*) (💻)

Implement the $\mathcal{O}(n \log n)$ divide-and-conquer approach described in Section 5.4 of *Kleinberg and Tardos*. Include your implementation in `problem_2/b.py`.

(c) (*10 points*) (💻)

Implement the $\mathcal{O}(n)$ randomized approach described in Section 13.7 of *Kleinberg and Tardos*. Include your implementation in `problem_2/c.py`.

(d) (*5 points*) (🖼️)

Perform an empirical analysis of the three approaches, plotting their runtime performance for a series of problem sizes.

Code input and output format. The input is a list of tuples or lists of floats corresponding to points in the plane. The output is a list containing the two closest points. For example, `closest_pair`$([(0,0),(1,1),(5,5)]) = [(0,0),(1,1)]$.

Problem 3.

Skip lists are randomized data structures that allow efficient expected-time insertion and search operations in a sorted list. We assume that list elements are always integers.

You will implement skip lists in this problem, as well as a linked-list baseline. We supply separate `Node` classes in each file, which will be the backbone of each implementation. Both implementations have the following interface:

- `insert(list, x: int) -> None`: add a new integer to the list.
- `search(list, x: int) -> Node`: return the node in the list that has value `x`.

For simplicity, assume that no repeated elements will be inserted into the list. Please read this resource on skip lists for more helpful information.

(a) (*6 points*) (📝💻)

As a baseline, implement a linked-list class. Give the runtime complexity of search and insertion. The linked list must always be in sorted order and all elements will be integers. Include your implementation in `problem_3/a.py`.

(b) (*6 points*) (📝💻)

problems/skip-lists/b.tex Implement a skip-list class. Give the expected and worst-case runtime complexity of search and insertion. All list elements will be integers. Include your implementation in `problem_3/b.py`.

(c) (*3 points*) (🖼️)

Using your implementations, perform an empirical runtime analysis of search in skip lists versus the linked-list baseline. There is a script in the `helper/` subdirectory that may be helpful.

(d) (*2 points*) (📝)

Let's now think about the memory usage of our implementations. Give the space overhead of linked lists. Then, give the expected and worst-case space overhead of skip lists.

(e) (*3 points*) (🖼️)

problems/skip-lists/e.tex Using your implementations, perform an empirical space overhead analysis of skip lists versus the linked-list baseline. Space consumption can be measured as the number of nodes. There is a script in the `helper/` subdirectory that may be helpful.

Note: A deterministic alternative to skip lists are binary trees. However, in order for binary trees to remain performant, they must be balanced. Balanced binary tree implementations such as AVL and Red-Black trees have relatively complicated implementations compared to skip lists.